OPERATING SYSTEMS – ASSIGNMENT 1

Name: S. Sharvesh **Register No:** 192525458

TASK 10 – Measure and Compare (4 Marks)

```
Measure system calls for different buffer sizes using strace -c; user-mode and kernel-mode time using time; and the time to
```

Alternative Concept and Complexity Analysis: Compare 1,000 threads and processes for creation time and memory, explaining what each shares.

Evidence Expected: Measured results in tables, compared with expectations and explained.

ANSWER

This task requires real measurements on the student's Linux system. The commands below create reproducible evidence. Run them several times, record the machine specifications and use the median to reduce noise. Results vary with CPU, kernel, storage, compiler, background load and virtualisation.

10.1 System-call count and elapsed time

gcc -std=c11 -Wall -Wextra -O2 copy_file.c -o copy_file

strace -c ./copy_file

/usr/bin/time -v ./copy_file

```
Record the program's own read() counter separately from strace's whole-process summary. The expected application counts are
```

10.2 Process and thread creation benchmark

Save the following as create_bench.c. It measures the time to create and join 1,000 threads and the time to fork and wait for 1,000 child processes. The child immediately exits so the benchmark measures process creation/reaping more than useful work.

#define _POSIX_C_SOURCE 200809L

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <sys/wait.h>
#include <unistd.h>
#include <time.h>
```

#define N 1000

static double seconds(struct timespec a, struct timespec b) {

```
return (b.tv_sec - a.tv_sec) +
```

(b.tv_nsec - a.tv_nsec) / 1e9;

```
}
static void *thread_fn(void *p) { (void)p; return NULL; }
int main(void) {
```

struct timespec a, b;

pthread_t threads[N];

clock_gettime(CLOCK_MONOTONIC, &a);

for (int i = 0; i < N; i++) {

int e = pthread_create(&threads[i], NULL, thread_fn, NULL);

```
if (e != 0) { fprintf(stderr, "pthread_create: %d\n", e); return 1; }
}
```

for (int i = 0; i < N; i++) pthread_join(threads[i], NULL);

clock_gettime(CLOCK_MONOTONIC, &b);

```
printf("1000 threads: %.6f seconds\n", seconds(a, b));
```

clock_gettime(CLOCK_MONOTONIC, &a);

for (int i = 0; i < N; i++) {

```
pid_t p = fork();
if (p < 0) { perror("fork"); return 1; }
```

if (p == 0) _exit(0);

int status;

```
if (waitpid(p, &status, 0) < 0) { perror("waitpid"); return 1; }
}
```

clock_gettime(CLOCK_MONOTONIC, &b);

```
printf("1000 processes: %.6f seconds\n", seconds(a, b));
return 0;
}
```

gcc -std=c11 -Wall -Wextra -O2 create_bench.c -pthread -o create_bench

./create_bench

/usr/bin/time -v ./create_bench

Expected qualitative result: threads are usually faster to create and use less incremental memory because they share one address space and process resources. Processes have separate virtual address spaces and stronger isolation; fork uses copy-on-write, so pages are not all copied immediately. The exact result must be measured.

10.3 Evaluation of measurements

```
For file copying, increasing buffer size should sharply reduce read() call count, while elapsed time should improve until s
```

QUESTION FROM THE PROVIDED ASSIGNMENT

Buffer size | Program read() count (expected) | strace read() count (your result) | User time | System time | Elapsed time

1 byte | 10001 | [enter measured] | [enter] | [enter] | [enter]

512 bytes | 21 | [enter measured] | [enter] | [enter] | [enter]

4096 bytes | 4 | [enter measured] | [enter] | [enter] | [enter]

65536 bytes (optional) | 2 (one data read + EOF) | [enter measured] | [enter] | [enter] | [enter]

Measurement | Your measured result

CPU model / RAM / Linux version | [enter system information]

1000 threads creation + join | [enter seconds]

1000 fork + wait child processes | [enter seconds]

Peak resident memory for thread test | [enter from time -v]

Peak resident memory for process test | [enter from time -v]